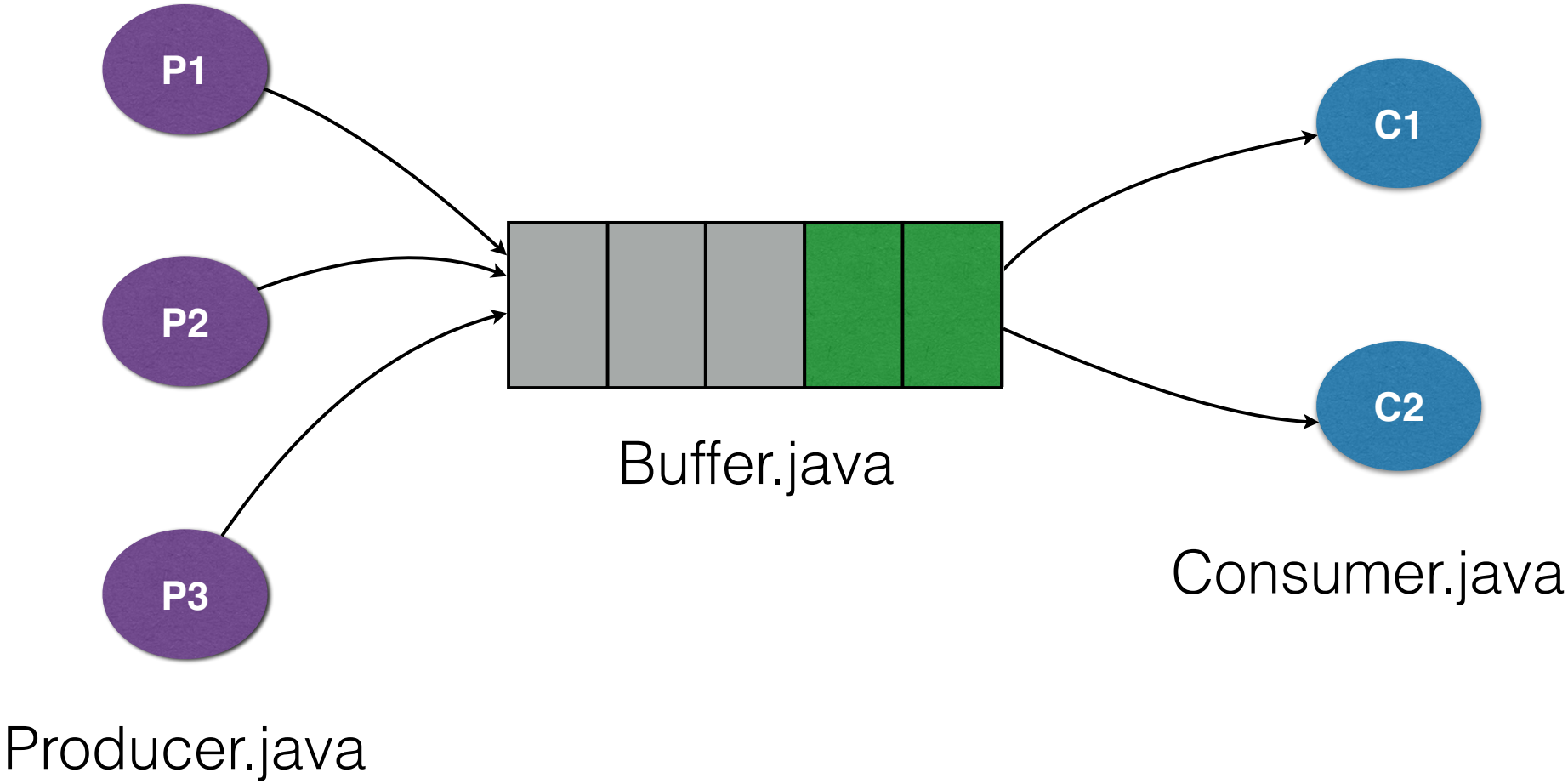
Producer-Consumer Protocol



Buffer.java

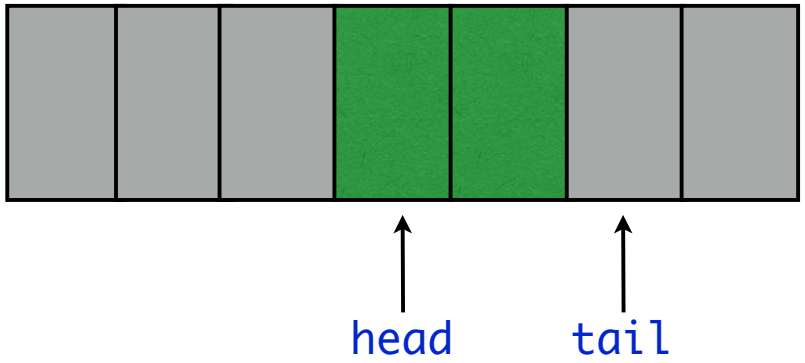
```
public class Buffer<E> {
    public final int capacity;
    private final E[] store;
    private int head, tail, size;

    public synchronized void put (E e) {
        while (isFull())
            wait();
        store[tail] = e;
        tail = next(tail);
        size++;
        notify();
    }

    public synchronized E get () {
        while (isEmpty())
            wait();
        E e = store[head];
        store[head] = null;
        head = next(head);
        size--;
        notify();
        return e;
    }

}
```

Circular vector



Buffer.java

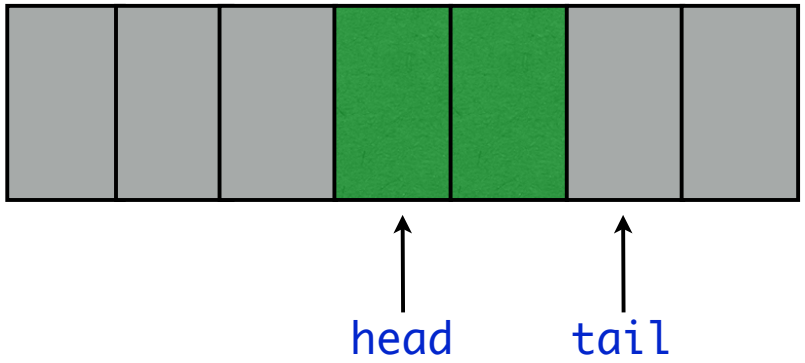
```
public class Buffer<E> {
    public final int capacity;
    private final E[] store;
    private int head, tail, size;

    public synchronized void put (E e) {
        while (isFull())
            wait();
        store[tail] = e;
        tail = next(tail);
        size++;
        notify();
    }

    public synchronized E get () {
        while (isEmpty())
            wait();
        E e = store[head];
        store[head] = null;
        head = next(head);
        size--;
        notify();
        return e;
    }

}
```

Circular vector



Producer.java

```
class Producer extends Thread {
    final Random rand = new Random();
    private int sleep;
    private Buffer<String> buffer;

    public Producer (int n, int sleep, Buffer<String> buffer) {
        super("producer."+n);
        this.sleep = sleep;
        this.buffer = buffer;
    }

    public void run () {
        while (true) {
            sleep(rand.nextInt(sleep));
            buffer.put(name);
        }
    }
}
```

Main.java

```
public class Main {

    public static void main (String[] args) throws Exception {
        int size = Integer.ParseInt(args[0]);
        int producers = Integer.ParseInt(args[1]);
        int consumers = Integer.ParseInt(args[2]);
        int sleepProducers = Integer.ParseInt(args[3]);
        int sleepConsumers = Integer.ParseInt(args[4]);
        final int threads = producers + consumers;
        final Buffer<String> buffer = new Buffer<String>(size);

        final Thread[] participants = new Thread[threads];

        int i = 0;
        for (int j=0; j<producers; j++)
            participants[i++] = new Producer(j, sleepProducers, buffer);
        for (int j=0; j<consumers; j++)
            participants[i++] = new Consumer(j, sleepConsumers, buffer);

        for (Thread t : participants)
            t.start();
    }
}
```

There is a bug in the code!

```
public class Main {

    public static void main (String[] args) throws Exception {
        int size = Integer.ParseInt(args[0]);
        int producers = Integer.ParseInt(args[1]);
        int consumers = Integer.ParseInt(args[2]);
        int sleepProducers = Integer.ParseInt(arg
        int sleepConsumers = Integer.ParseInt(
        final int threads = producers +
        final Buffer<String> buffe

        final Thread[] p                    eads];

        int i
        f                    +)
                    new Producer(j, sleepProducers, buffer);
                    nsumers; j++)
                    nts[i++] = new Consumer(j, sleepConsumers, buffer);

        f  (Thread t : participants)
            t.start();
    }
}
```

Occasionally the system deadlocks!

It reaches a state where all threads are blocked

How to test concurrent code?

- Errors in concurrent programs are difficult to reproduce
  - Simulate nondeterministic interleaving of threads
  - Rely on luck :-(

Extract execution traces: method **put**

```
public synchronized void put (E e) {
    while (isFull())
        wait();
    store[tail] = e;
    tail = next(tail);
    size++;
    notify();
}
```

Extract execution traces: method **put**

```java
public synchronized void put (E e) {
    while (isFull())
        wait();
    store[tail] = e;
    tail = next(tail);
    size++;
    notify();
}
```

```java
public synchronized void put (E e)  {
        String name = Thread.currentThread().getName();
        while (isFull()) {
            logger.info(String.format("buffer full; %s waits", name));
            waitSetSize++;
            wait();
            logger.info(String.format("%s is notified", name));
        }
        logger.info(String.format("%s successfully puts", name));
        if (waitSetSize > 0)
            waitSetSize--;
        logger.fine(String.format("wait set size is %d", waitSetSize));
        store[tail] = e;
        tail = next(tail);
        size++;
        lastChange = System.currentTimeMillis();
        notify();
    }
```

Extract execution traces: method **put**

```
public synchronized void put (E e) {
    while (isFull())
        wait();
    store[tail] = e;
    tail = next(tail);
    size++;
    notify();
}
```

```
publi
                                    hread().getName();

                                    String.format("buffer full; %s waits", name));
                                    size++;
                                    wait();
                                    logger.info(String.format("%s is notified", name));
    }
    logger.info(String.format("%s successfully puts", name));
    if (waitSetSize > 0)
        waitSetSize--;
    logger.fine(String.format("wait set size is %d", waitSetSize));
    store[tail] = e;
    tail = next(tail);
    size++;
    lastChange = System.currentTimeMillis();
    notify();
}
```

Annotate method get and Main class

Testing the protocol

Buffer entries	Producers	Consumers	Found a deadlock
10	5	5	NO

Setup (MacBook Pro)
2.6 GHz Intel Core i5
8 GB of RAM
—> executed each test for several hours

Testing the protocol

Buffer entries	Producers	Consumers	Found a deadlock
10	5	5	NO
5	10	3	NO

Setup (MacBook Pro)
2.6 GHz Intel Core i5
8 GB of RAM
—> executed each test for several hours

Testing the protocol

Buffer entries	Producers	Consumers	Found a deadlock
10	5	5	NO
5	10	3	NO
2	3	2	NO

Setup (MacBook Pro)
2.6 GHz Intel Core i5
8 GB of RAM
—> executed each test for several hours

Testing the protocol

Buffer entries	Producers	Consumers	Found a deadlock
10	5	5	NO
5	10	3	YES
2	3	2	NO

Setup (64 node cluster)
64-core node (4 AMD Opteron 6272)
64 GB of RAM
2 NIC Intel Corporation 82576 Gigabit Network Connection

Analyse the trace of 2nd Test

- The trace of the 2nd test has 101 lines :-(

```
DEADLOCK
14:17:01.589: consumer.1 is notified
14:17:01.589: buffer empty; consumer.1 waits
14:17:01.590: producer.7 successfully puts
14:17:01.590: consumer.1 is notified
14:17:01.590: consumer.1 successfully gets
14:17:01.590: producer.1 successfully puts
14:17:01.590: producer.5 successfully puts
14:17:01.590: consumer.0 successfully gets
14:17:01.591: consumer.2 successfully gets
14:17:01.591: producer.5 successfully puts
...
14:17:01.646: buffer full; producer.3 waits
14:17:01.646: producer.8 is notified
14:17:01.646: buffer full; producer.8 waits
14:17:01.646: producer.0 is notified
14:17:01.646: buffer full; producer.0 waits
14:17:01.647: buffer full; producer.4 waits
14:17:38.105: DEADLOCK after 48470 messages and 23.5 seconds!
```

Analyse the trace

- Producers and consumers wait on the same wait set
- When method `get` calls `notify()`
  - It should notify a producer (buffer has a slot available)
  - Occasionally it notifies a consumer
- When method `put` calls `notify()`
  - It should notify a consumer (buffer has data available)
  - Occasionally it notifies a producer

Analyse the trace

- Producers and consumers wait on the same wait set
- When method `get` calls `notify()`
  - It should notify a producer (buffe[illegible] available)
  - Occasionally it notifies [illegible]er
- When metho[illegible] `notify()`
  - [illegible] notify a consumer (buffer has data [illegible]ailable)
  - Occasionally it notifies a producer

May lead to a deadlock!

Next steps…

- Fix the Java code by replacing every call to `notify()` by a call to `notifyAll()`
- Execute all tests again

Next steps…

- Fix the Java code by replacing every call to `notify()` by a call to `notifyAll()`
- Execute all tests again

Is it enough?